

Artificial Intelligence

Course Information

- **Course Code:** CSC462 Course
- **Title:** Artificial Intelligence
- **Credit Hours:** 4 (3,1)

Text and Reference Books

- **Textbook:** Artificial Intelligence: A Modern Approach, Russell, S., and Norvig, P., Pearson, 2020.
- **Reference Book:** Artificial Intelligence Basics: A Non-Technical Introduction, Taulli, T., Apress, 2019.



Artificial Intelligence Search Agents

Informed search

Topics Covered: Adversarial Search: Game Theory, Heuristic, Min-Max Procedure & Alpha-Beta Pruning

Adversarial search and Games

- Adversarial search is a search, where we examine the problem which arises when we try to plan ahead of the world and other agents are planning against us.
- In previous topics, we have studied the search strategies which are only associated with a single agent that aims to find the solution which often expressed in the form of a sequence of actions.
- But there might be some situations where more than one agent is searching for the solution in the same search space, and this situation usually occurs in game playing.

Adversarial Search

- The environment with more than one agent is termed as multi-agent environment, in which each agent is an opponent of other agent and playing against each other. Each agent needs to consider the action of other agent and effect of that action on their performance.
- Searches in which two or more players with conflicting goals are trying to explore the same search space for the solution, are called adversarial searches, often known as Games.
- Backing algorithm, best move strategy used, max will try to maximize its utility (best move), min will try to minimize utility(worst move)

Types of Games in AI

- Perfect information: A game with the perfect information is that in which agents can look into the complete board. Agents have all the information about the game, and they can see each other moves also. Examples are Chess, Checkers, Go, etc.
- Imperfect information: If in a game agents do not have all information about the game and not aware with what's going on, such type of games are called the game with imperfect information, such as tic-tac-toe, Battleship, blind, Bridge, etc.

Types of Games in AI

- Deterministic games: Deterministic games are those games which follow a strict pattern and set of rules for the games, and there is no randomness associated with them. Examples are chess, Checkers, Go, tic-tac-toe, etc.
- Non-deterministic games: Non-deterministic are those games which have various unpredictable events and has a factor of chance or luck. This factor of chance or luck is introduced by either dice or cards. These are random, and each action response is not fixed. Example: Backgammon, Monopoly, Poker, etc.

Zero-Sum Game

- Zero-sum games are adversarial search which involves pure competition.
- In Zero-sum game each agent's gain or loss of utility is exactly balanced by the losses or gains of utility of another agent.
- One player of the game try to maximize one single value, while other player tries to minimize it.
- Each move by one player in the game is called as ply.
- Chess and tic-tac-toe are examples of a Zero-sum game.

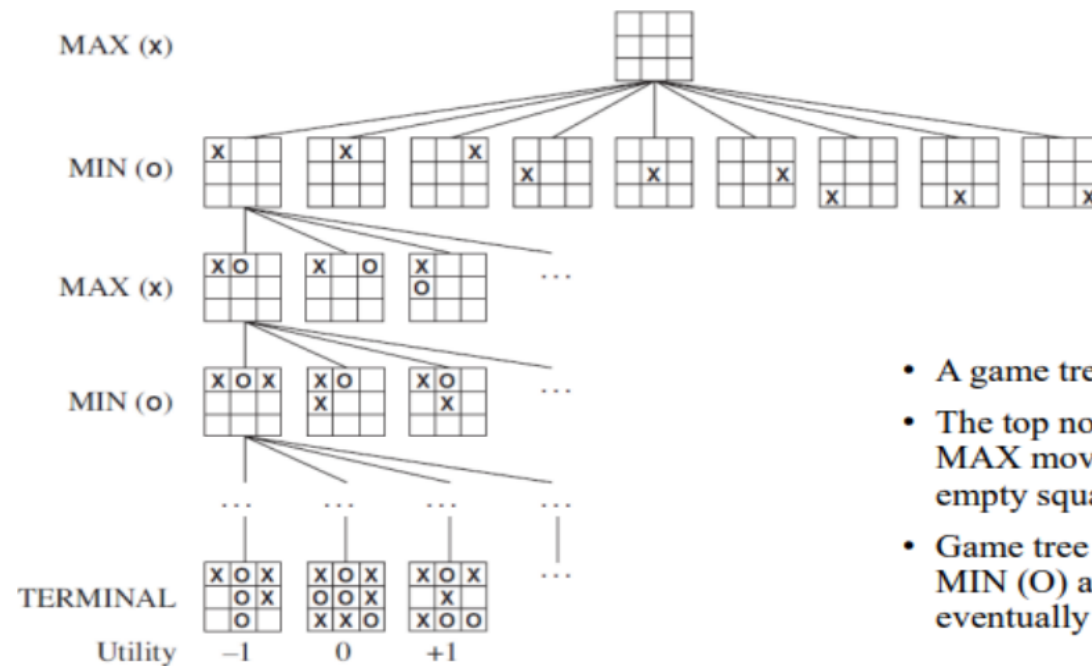
Games as Search Problem

- Game Problem Formalization: A game can be formally defined as a kind of search problem with the following components:
- Initial state: It specifies how the game is set up at the start.
- Player(s): It specifies which player has moved in the state space. Action(s): It returns the set of legal moves in state space.
- Result: It is the transition model, which specifies the result of moves in the state space.
- Terminal-Test(s): Terminal test is true if the game is over, else it is false at any case. The state where the game ends is called terminal states.

Games as Search Problem

- Utility(s, p): A utility function gives the final numeric value for a game that ends in terminal states s for player p . It is also called payoff function. For Chess, the outcomes are a win, loss, or draw and its payoff values are +1, 0, $\frac{1}{2}$. And for tic-tac-toe, utility values are +1, -1, and 0.

Game Tree



- A game tree for the game of tic-tac-toe.
- The top node is the initial state, and MAX moves first, placing an X in an empty square.
- Game tree gives alternating moves by MIN (O) and MAX (X), until eventually reaching terminal states.

Mini-Max Algorithm in Artificial Intelligence

- Mini-max algorithm is a recursive or backtracking algorithm which is used in decision-making and game theory.
- Mini-Max algorithm uses recursion to search through the game-tree.
- Min-Max algorithm is mostly used for game playing in AI. Such as Chess, Checkers, tictac-toe, go, and various tow players game. This Algorithm computes the minimax decision for the current state.
- In this algorithm two players play the game; one is called MAX and other is called MIN.
- Both the players fight it as the opponent player gets the minimum benefit while they get the maximum benefit.
- Both Players of the game are opponent of each other, where MAX will select the maximized value and MIN will select the minimized value.

Mini -Max Algorithm in Artificial Intelligence

- The minimax algorithm performs a depth -first search algorithm for the exploration of the complete game tree.
- The minimax algorithm proceeds all the way down to the terminal node of the tree, then backtrack the tree as the recursion.

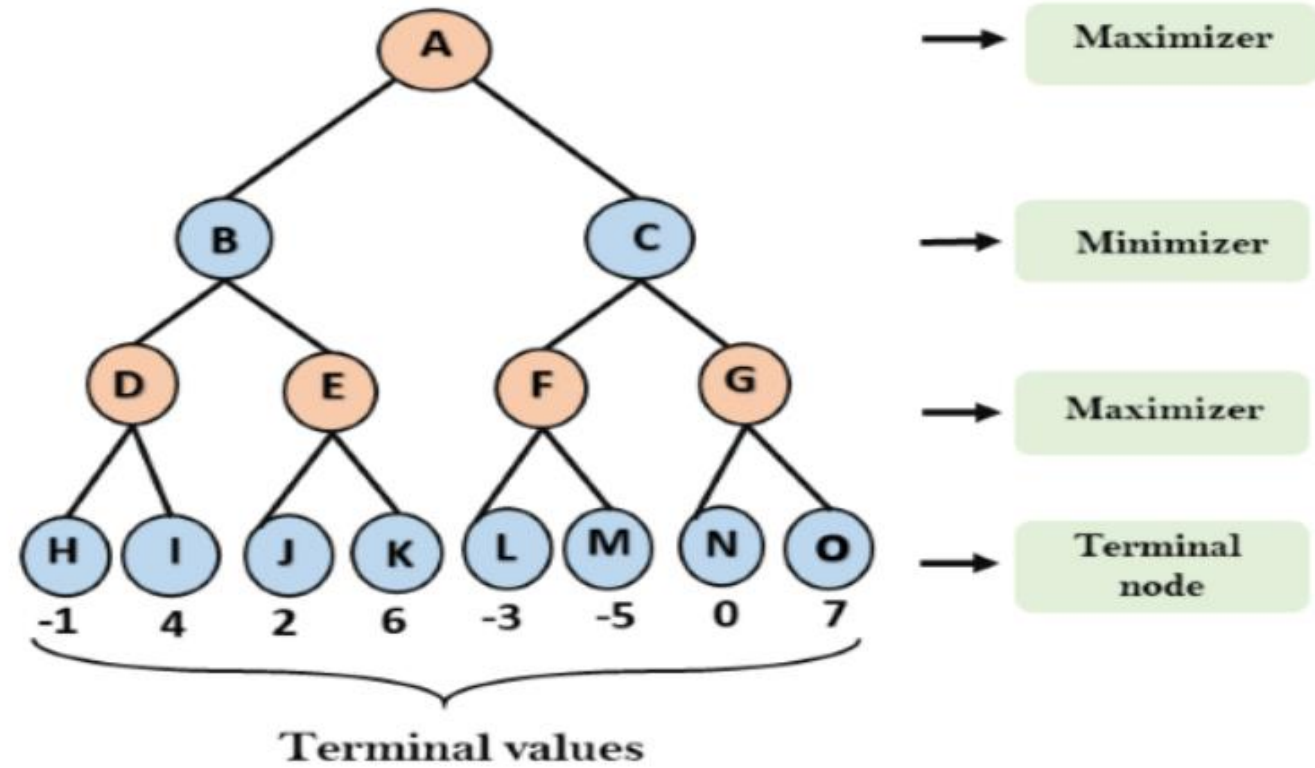
Working of Min-Max Algorithm

- we have taken an example of game-tree which is representing the two-player game.
- In this example, there are two players one is called Maximizer and other is called Minimizer.
- Maximizer will try to get the Maximum possible score, and Minimizer will try to get the minimum possible score.
- This algorithm applies DFS, so in this game-tree, we have to go all the way through the leaves to reach the terminal nodes.
- At the terminal node, the terminal values are given so we will compare those value and backtrack the tree until the initial state occurs.

Following are the main steps involved in solving the two-player game tree:

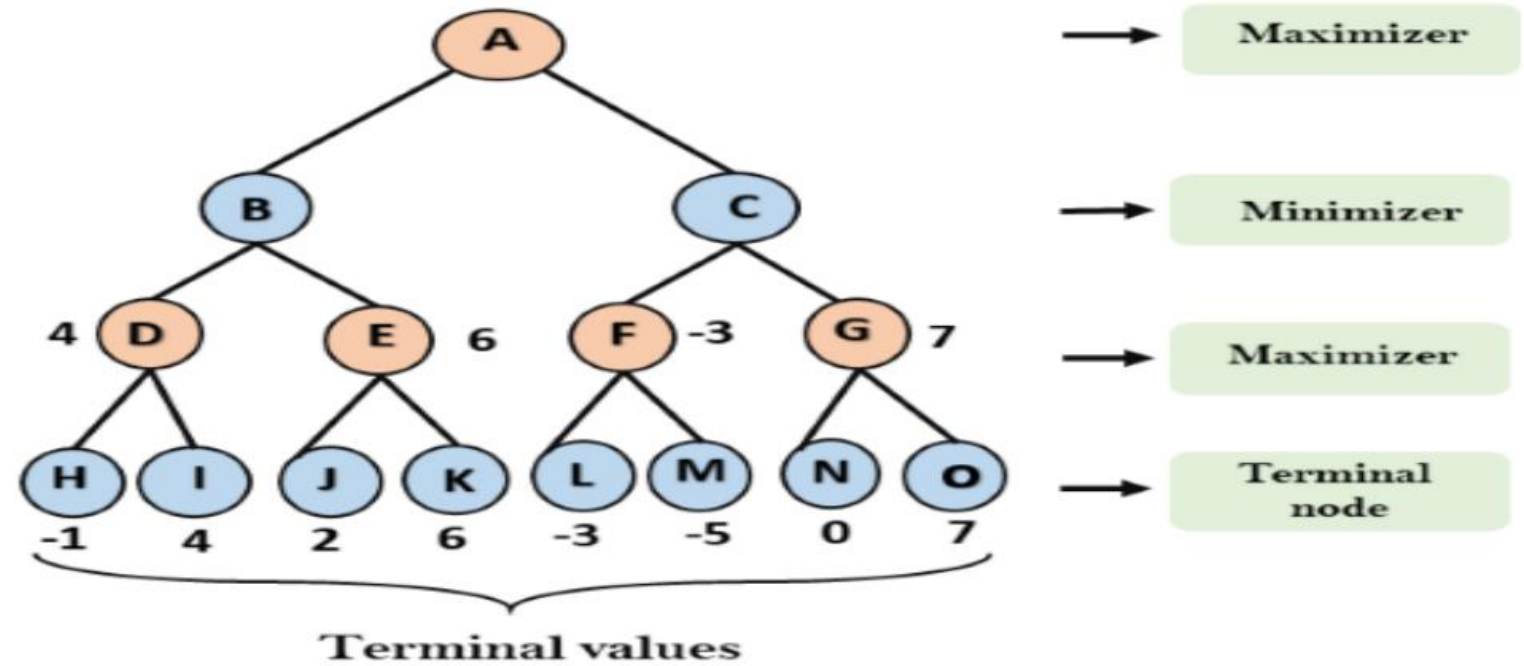
- In the first step, the algorithm generates the entire game-tree and apply the utility function to get the utility values for the terminal states. In the below tree diagram, let's take A is the initial state of the tree. Suppose maximizer takes first turn which has worst-case initial value = $-\infty$, and minimizer will take next turn which has worst-case initial value = $+\infty$.

Step1:



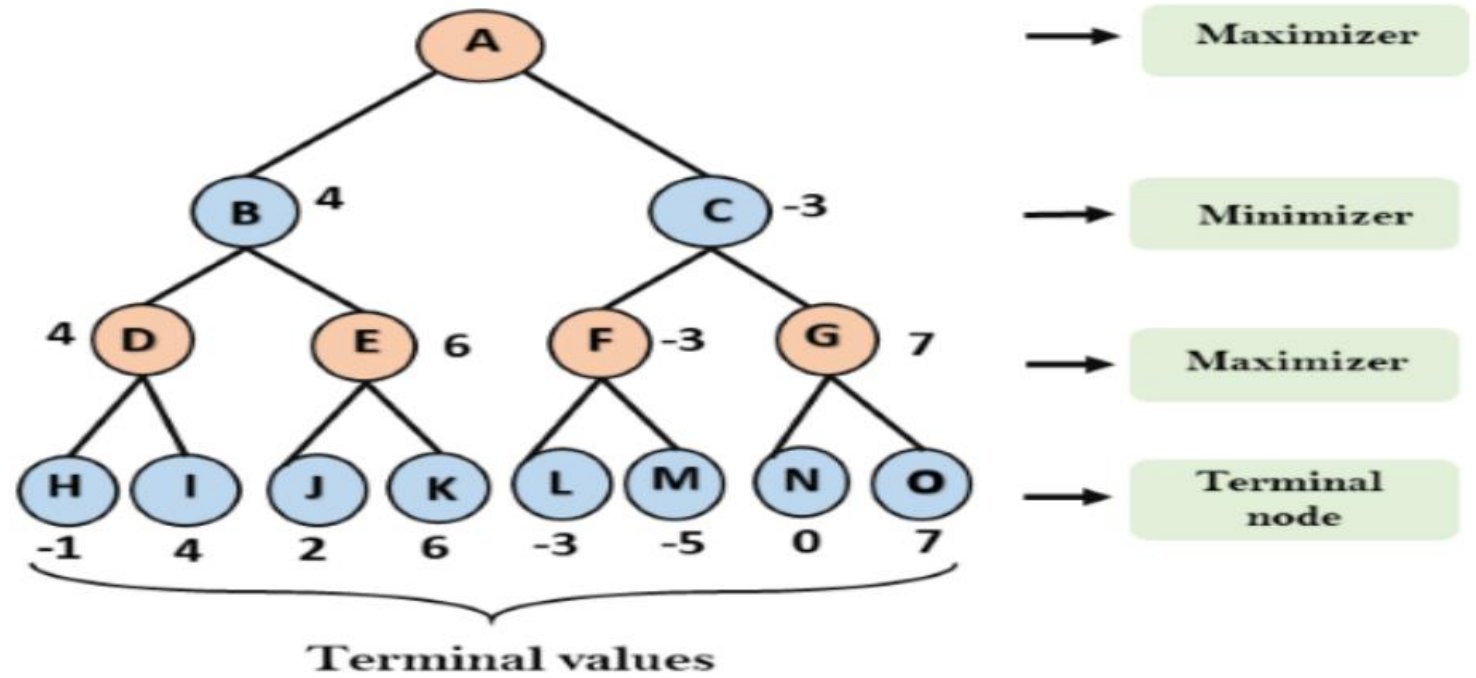
- Now, first we find the utilities value for the Maximizer, its initial value is $-\infty$, so we will compare each value in terminal state with initial value of Maximizer and determines the higher nodes values. It will find the maximum among the all. • For node D $\max(-1, -\infty) \Rightarrow \max(-1, 4) = 4$ • For Node E $\max(2, -\infty) \Rightarrow \max(2, 6) = 6$ • For Node F $\max(-3, -\infty) \Rightarrow \max(-3, -5) = -3$ • For node G $\max(0, -\infty) = \max(0, 7) = 7$

Step 2:



- Step 3: In the next step, it's a turn for minimizer, so it will compare all nodes value with $+\infty$, and will find the 3rd layer node values. For node B= $\min(4,6) = 4$ For node C= $\min(-3, 7) = -3$

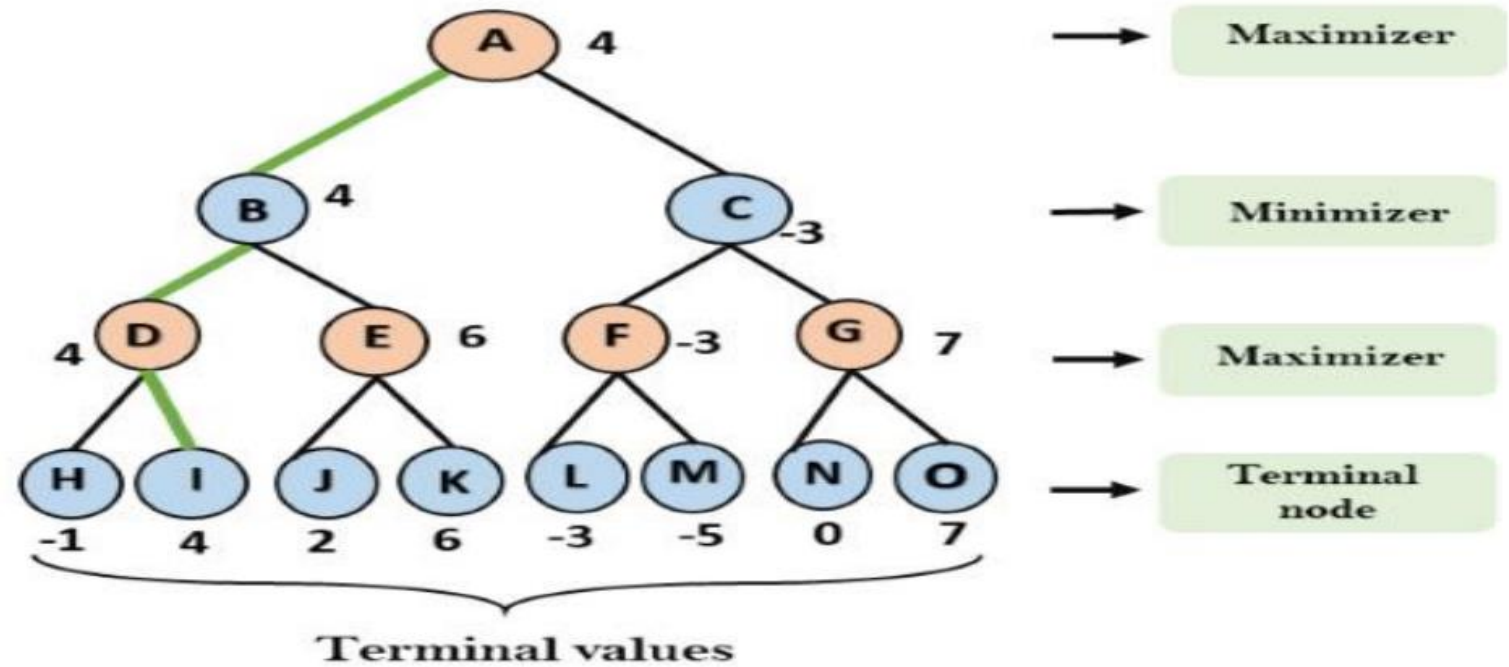
Step 3:



Step 4:

- Now it's a turn for Maximizer, and it will again choose the maximum of all nodes value and find the maximum value for the root node. In this game tree, there are only 4 layers, hence we reach immediately to the root node, but in real games, there will be more than 4 layers. For node A $\max(4, -3) = 4$

Step 4:



Pseudo-code for MinMax Algorithm

- function minimax(node, depth, maximizingPlayer) is
- if depth ==0 or node is a terminal node then
- return static evaluation of node
-
- if MaximizingPlayer then // for Maximizer Player
- maxEva= -infinity
- for each child of node do
- eva= minimax(child, depth-1, false)
- maxEva= max(maxEva, eva) //gives Maximum of the values
- return maxEva
-
- else // for Minimizer player
- minEva= +infinity
- for each child of node do
- eva= minimax(child, depth-1, true)
- minEva= min(minEva, eva) //gives minimum of the values
- return minEva

Alpha-Beta Pruning

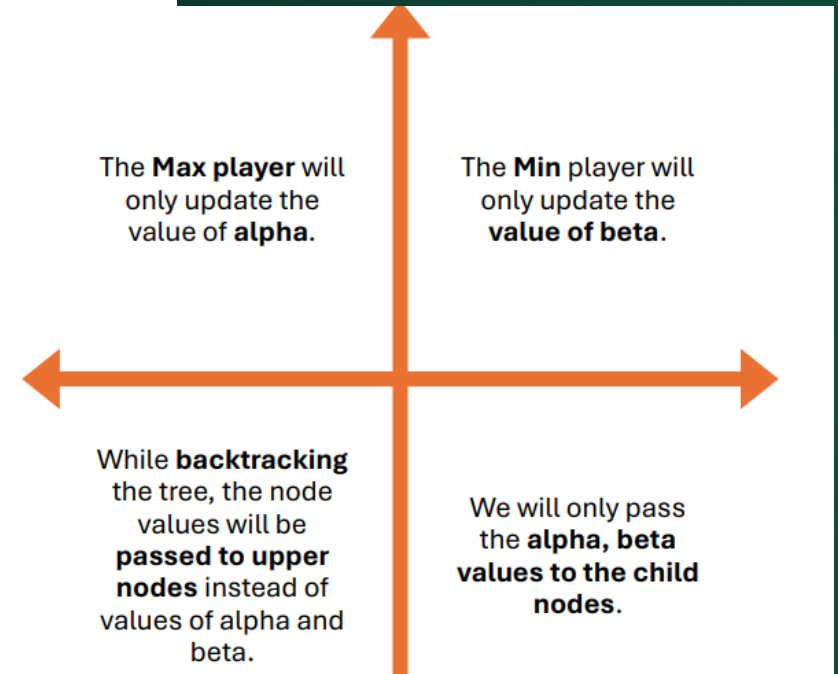
- Alpha-beta pruning is a modified version of the minimax algorithm. It is an optimization technique for the minimax algorithm.
- In the minimax search algorithm that the number of game states it has to examine are exponential in depth of the tree.
- Since we cannot eliminate the exponent, but we can cut it to half. Hence there is a technique by which without checking each node of the game tree we can compute the correct minimax decision, and this technique is called pruning.
- This involves two threshold parameter Alpha and beta for future expansion, so it is called alphabeta pruning. It is also called as Alpha-Beta Algorithm.

Alpha-Beta Pruning

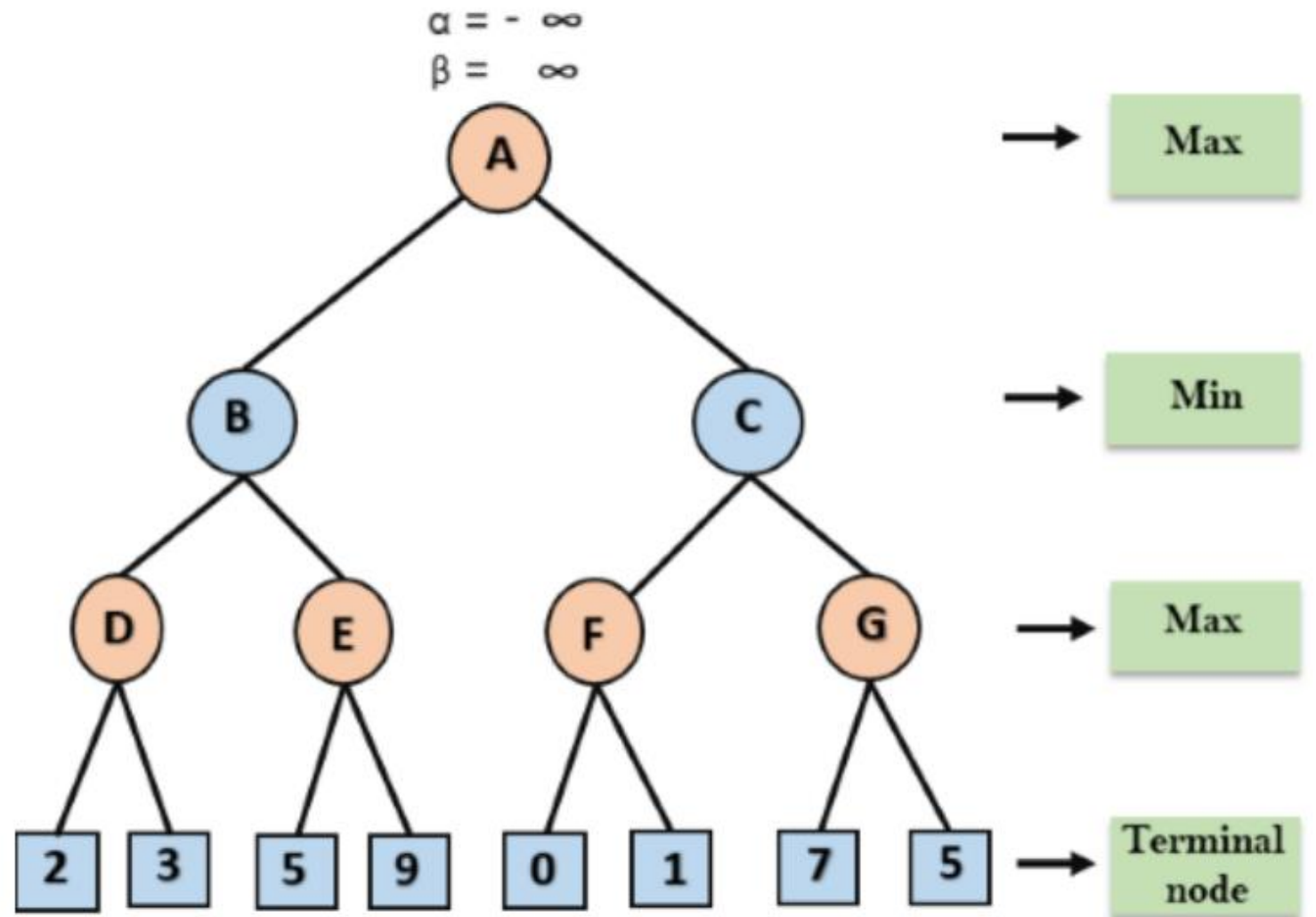
- The two-parameter can be defined as:
- Alpha: The best (highestvalue) choice we have found so far at any point along the path of Maximizer. The initial value of alpha is $-\infty$.
- Beta: The best (lowestvalue) choice we have found so far at any point along the path of Minimizer. The initial value of beta is $+\infty$.

Condition for Alpha-beta pruning:

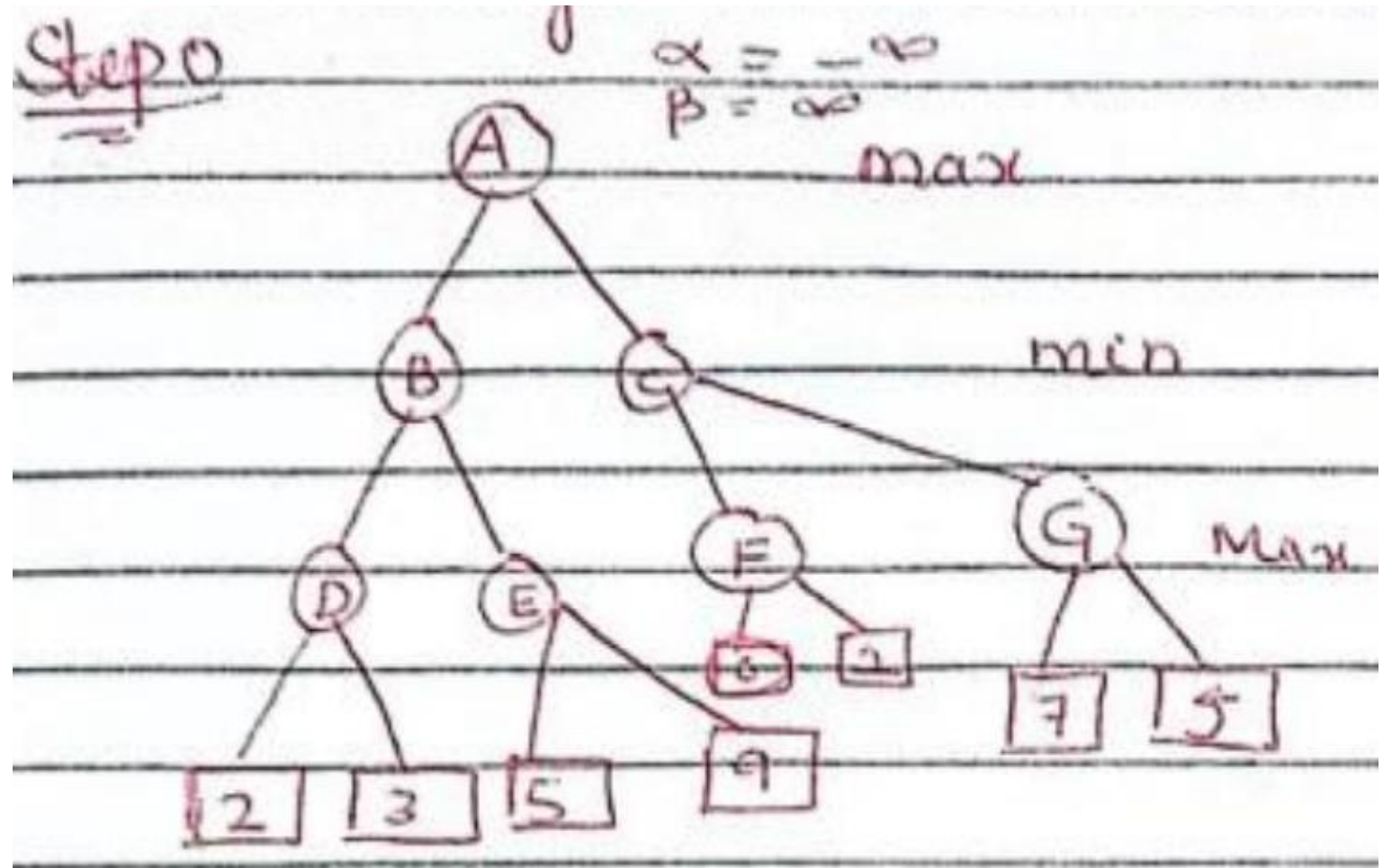
- The main condition which required for alpha-beta pruning is:
- $\alpha \geq \beta$
- Key points about alpha - beta pruning:



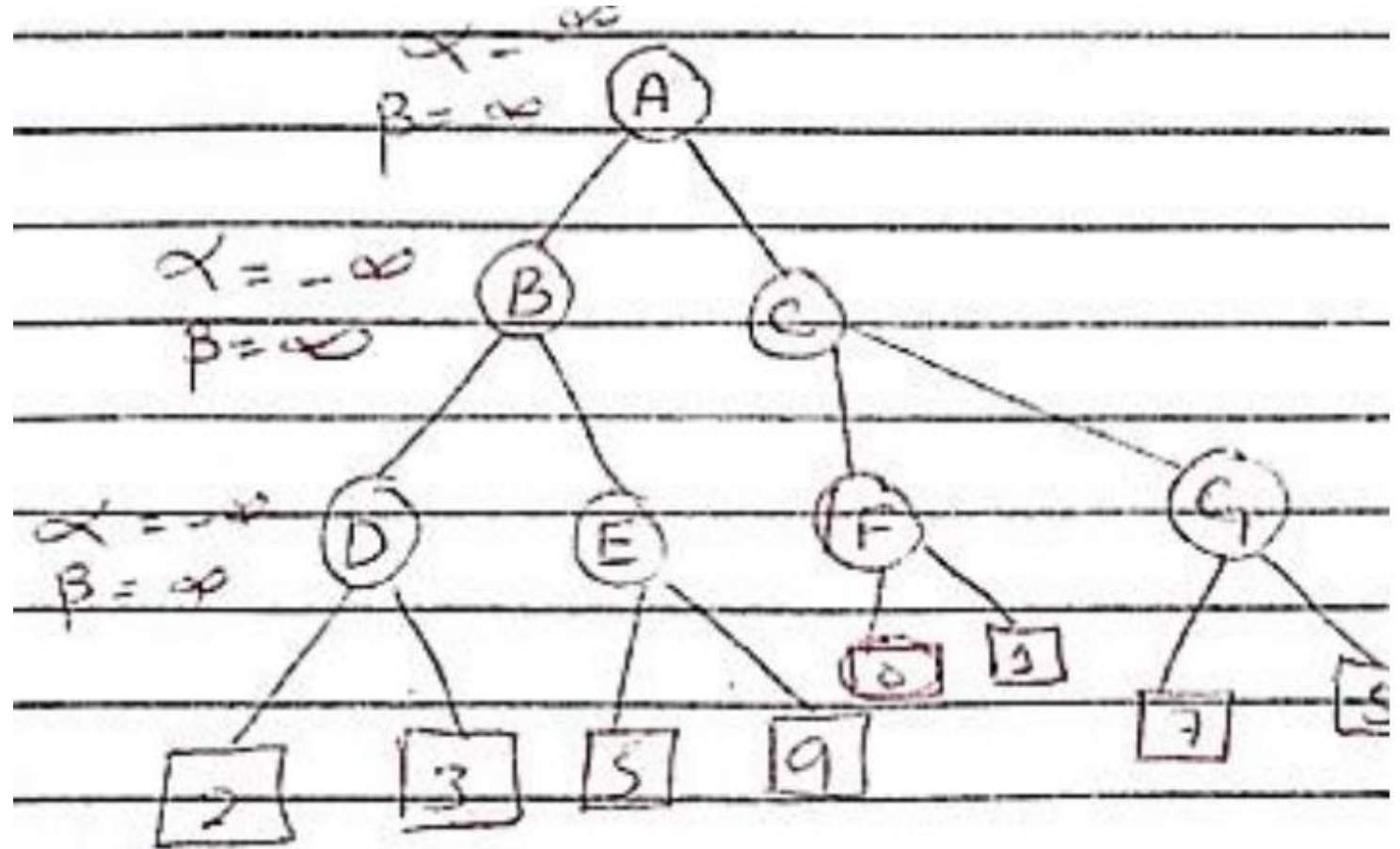
Working of Alpha-Beta Pruning:



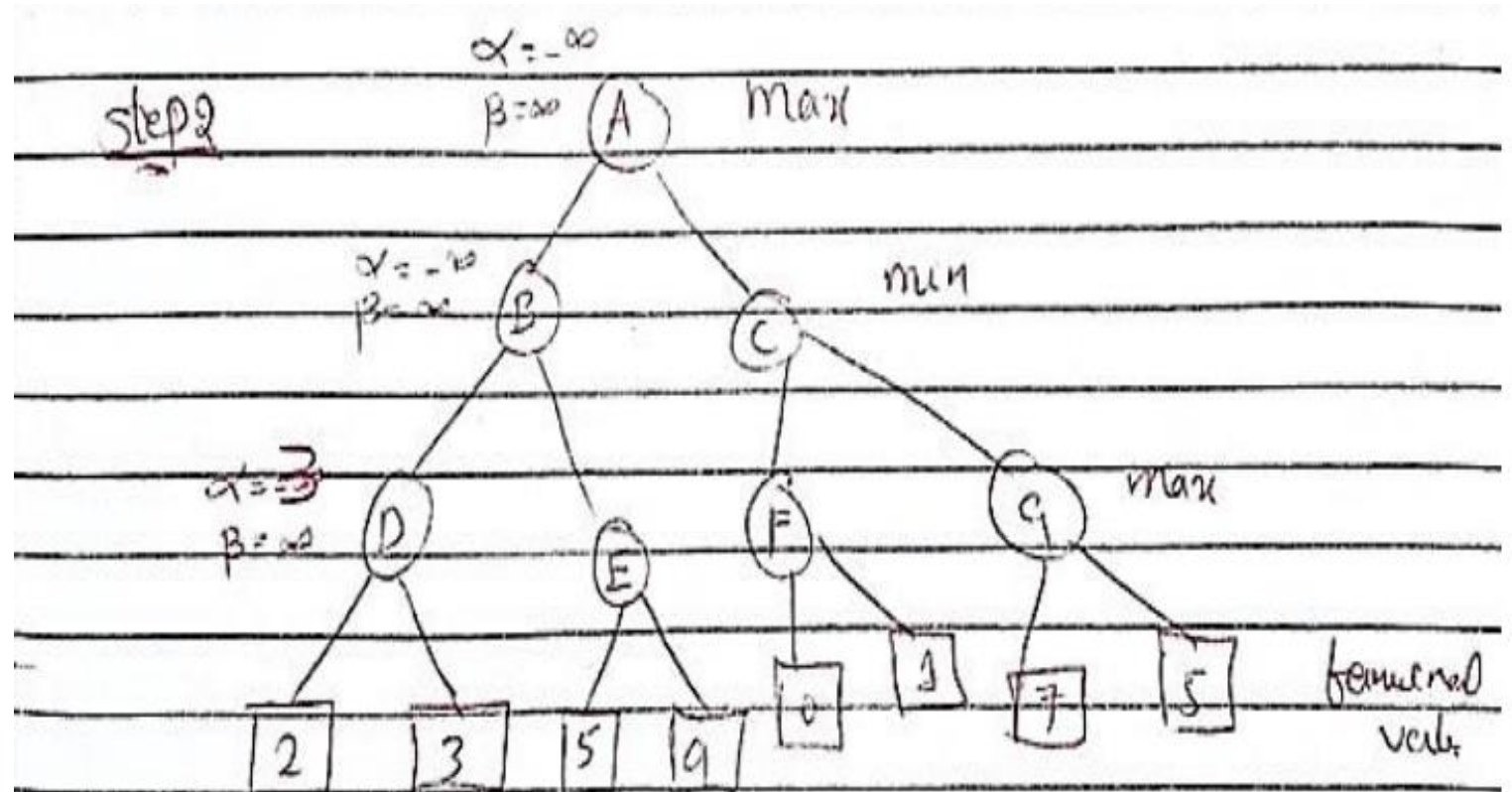
Step 0



Step 1



Step 2



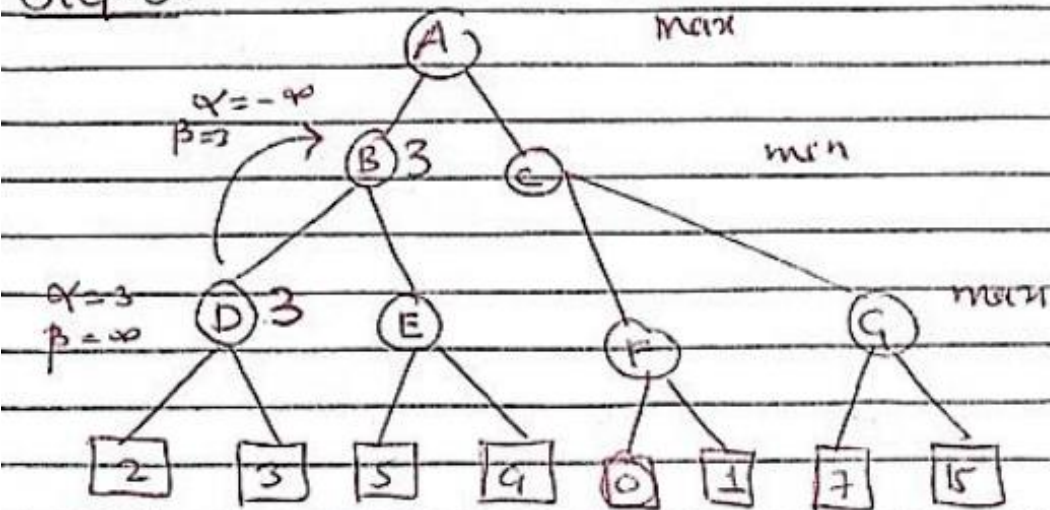
$$\max(2, -\infty) = 2$$

$$\max(2, 3) = 3$$

So node D value will also be 3, $\alpha = 3$

Step 3

step 3:

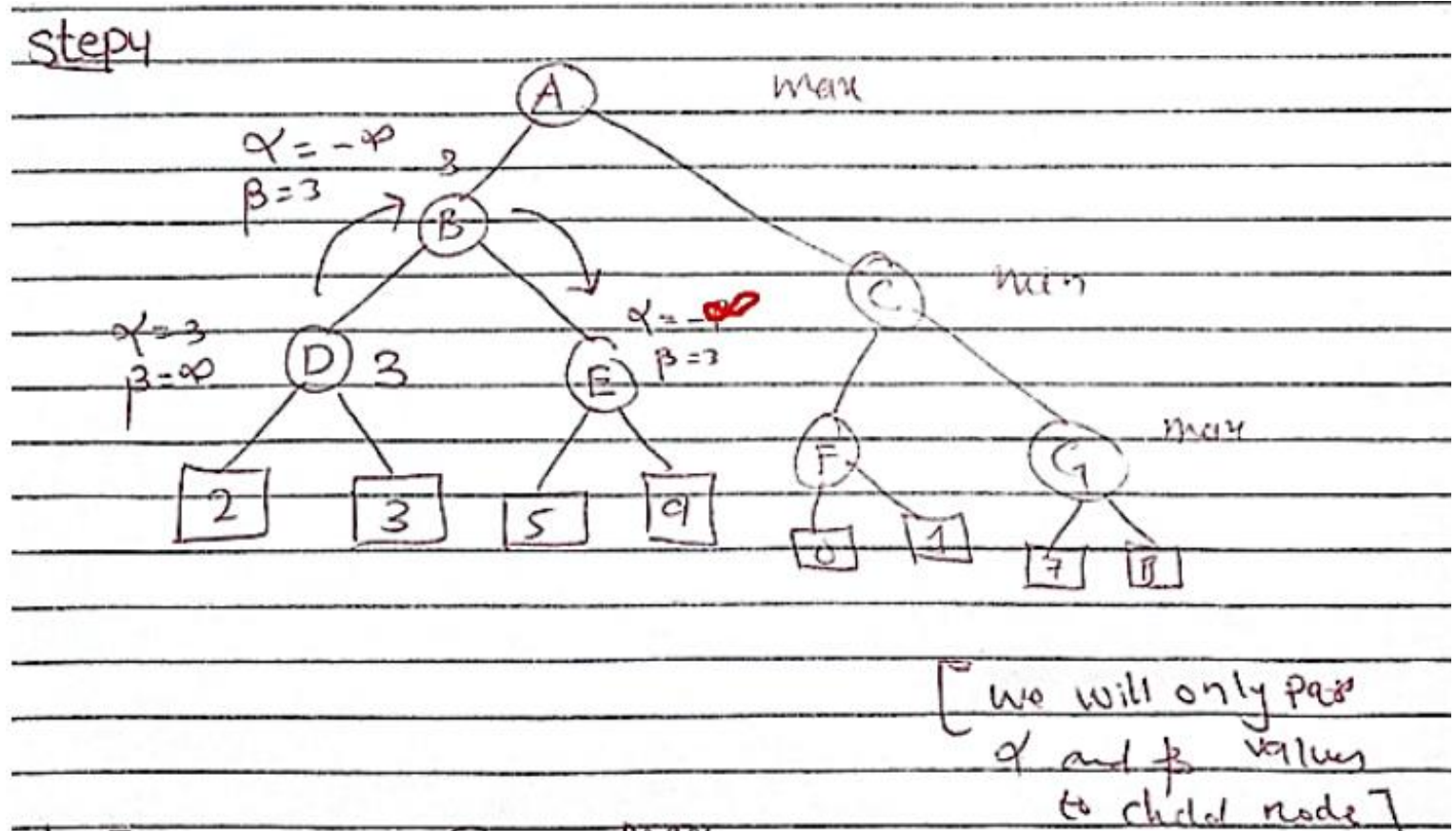


$$\alpha = -\infty$$

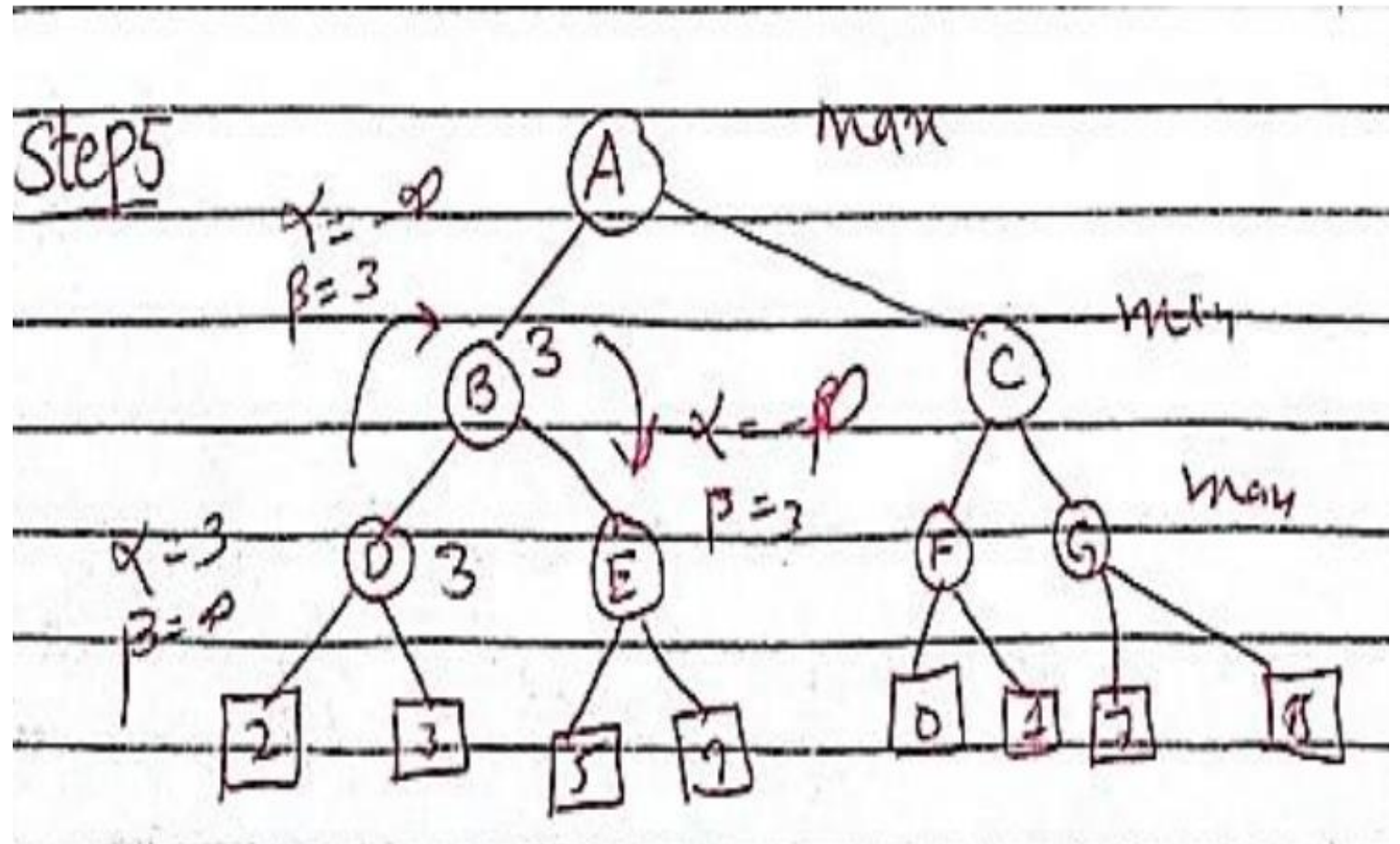
$$\beta = \min(3, \infty) = 3$$

[Backtracking, node values will be passed to upper node instead of value of α and β]

Step 4



Step 5



$$q' = -\infty$$

$$\max(5, -\infty) = 5$$

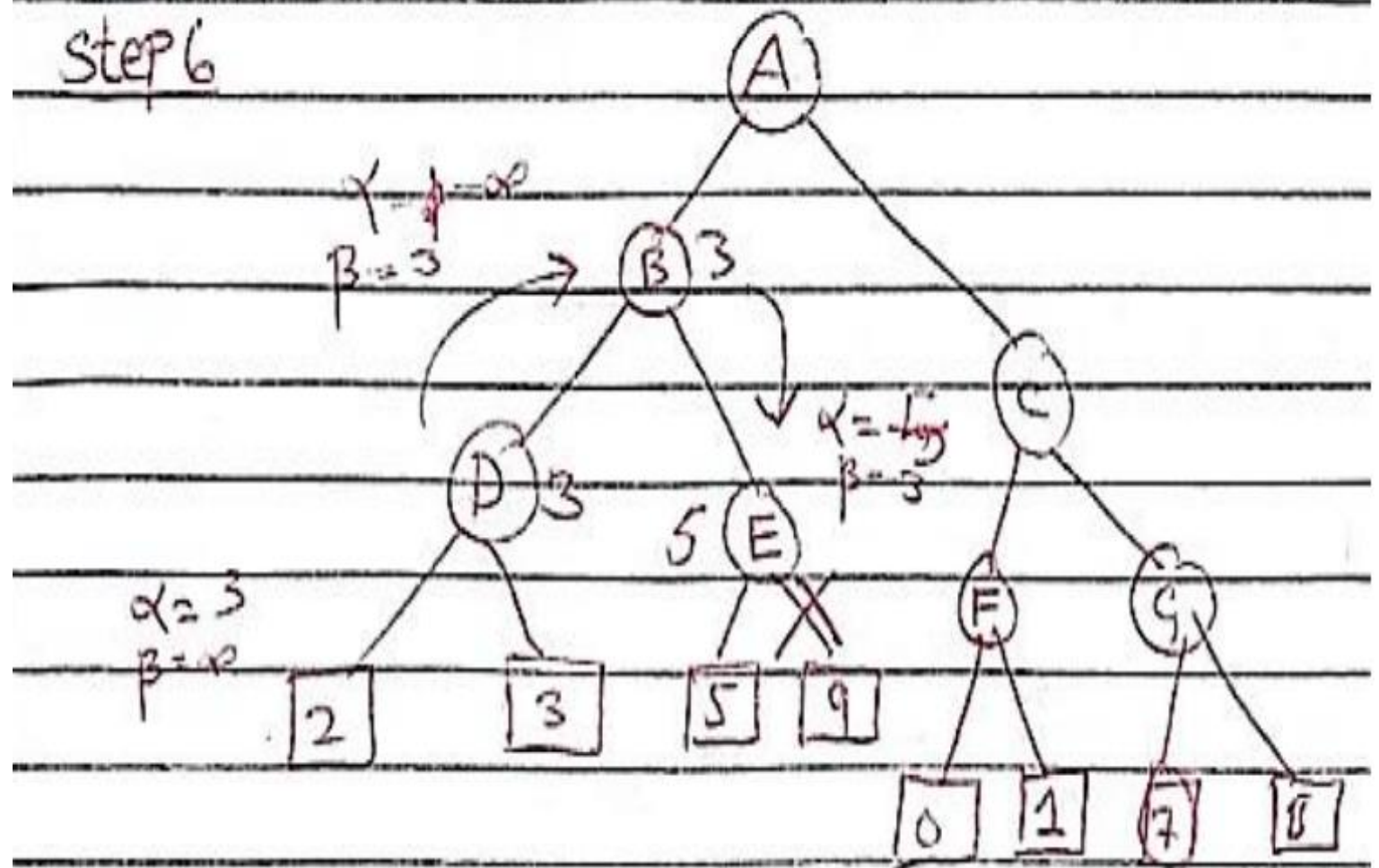
$$\boxed{\text{node} = 5}$$

$$\boxed{\alpha = 5}$$

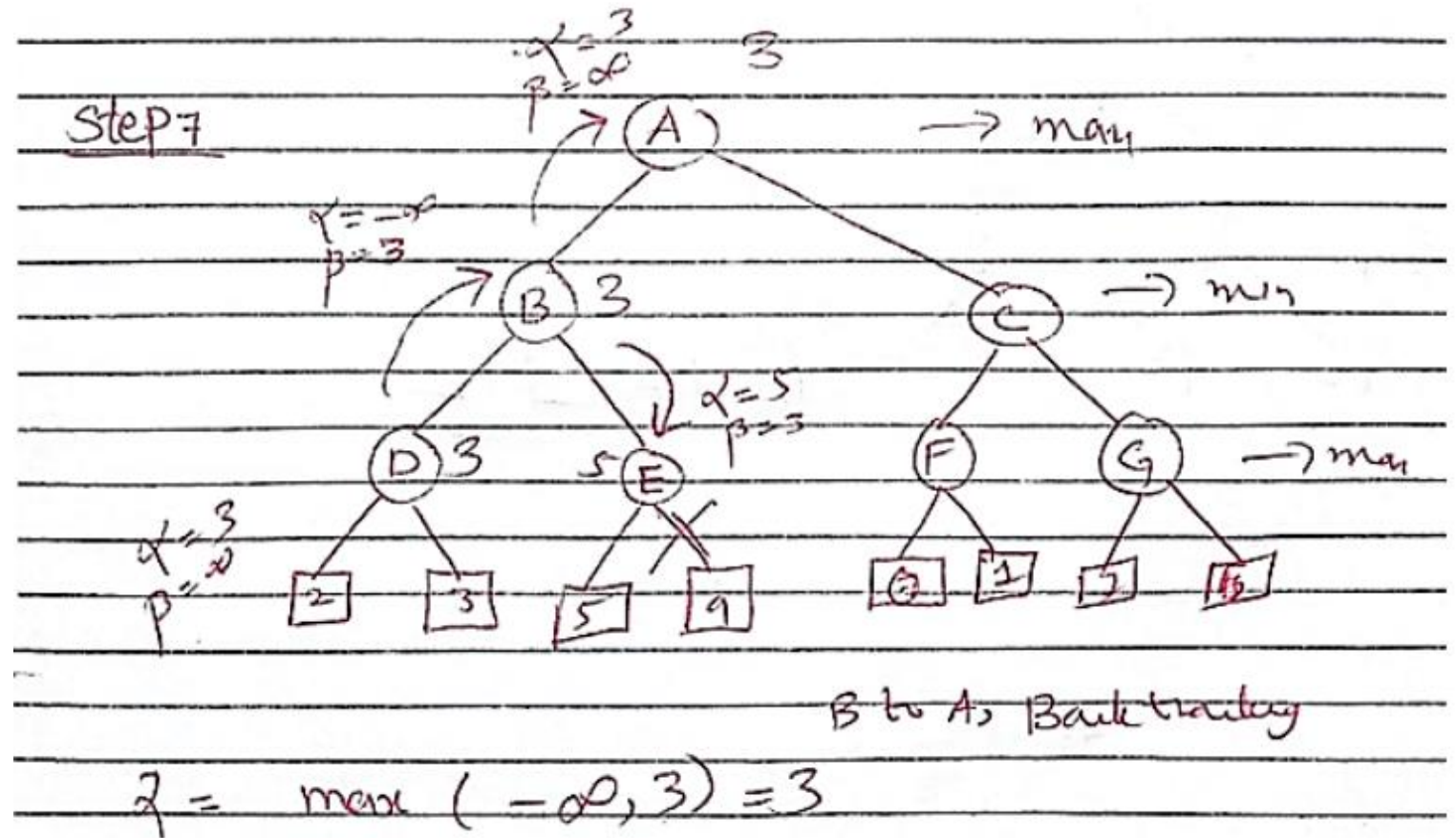
$$q > p$$

$$5 > 3$$

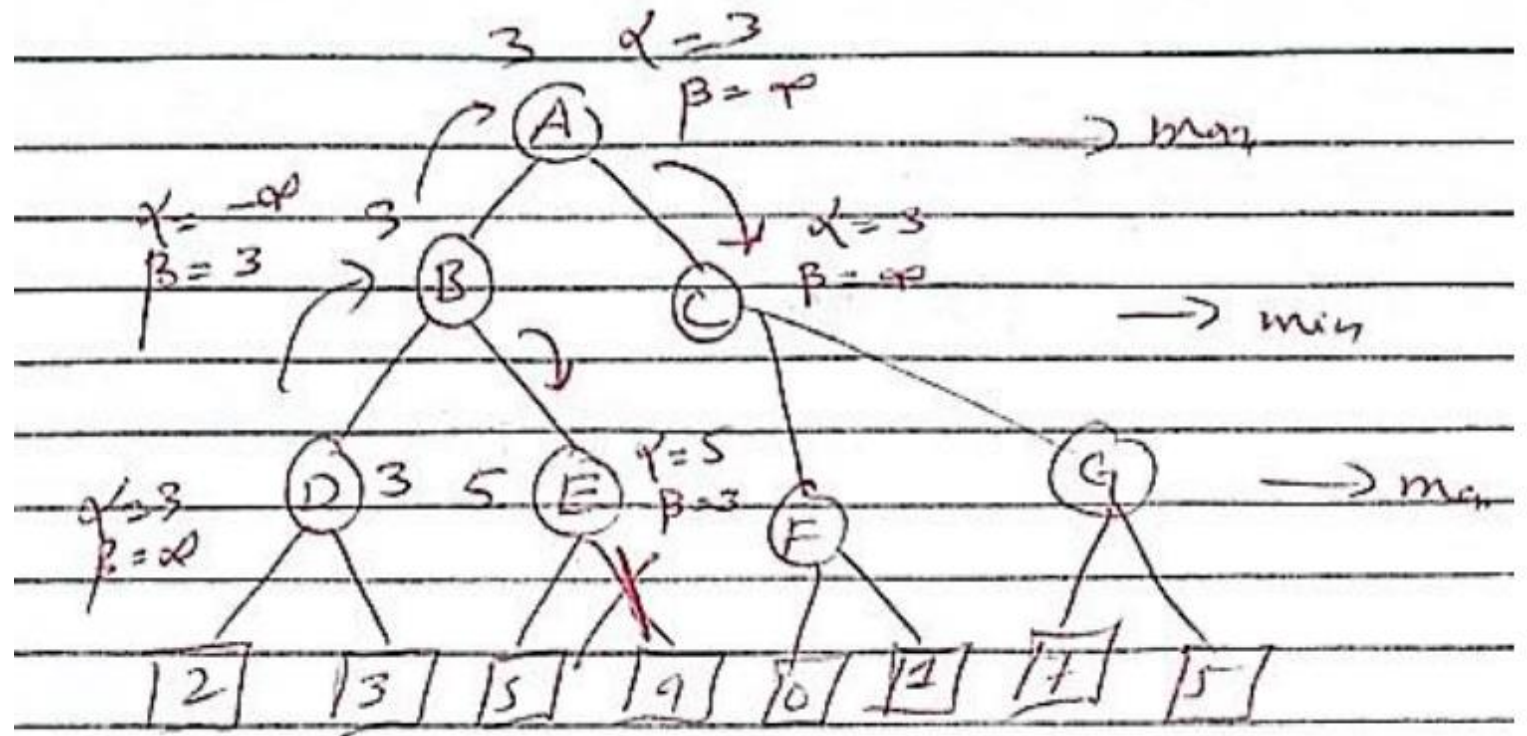
Step 6



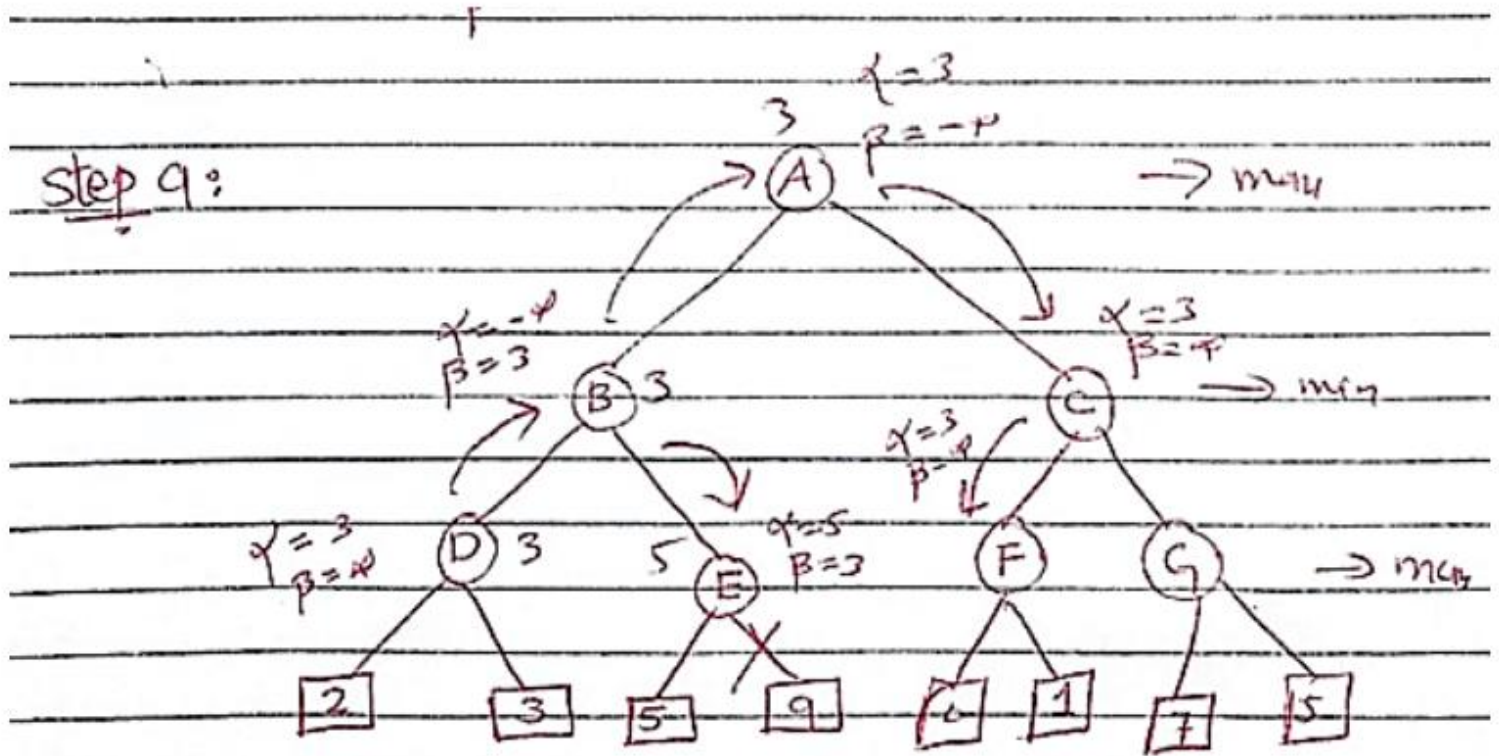
Step 7



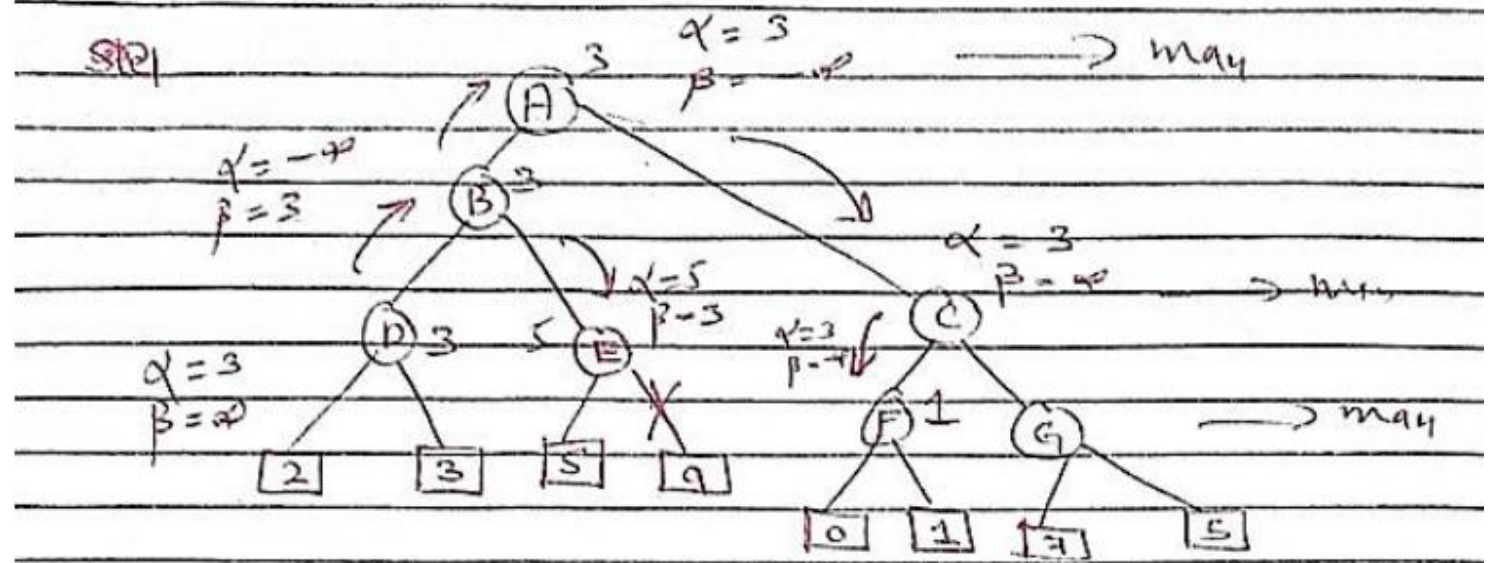
Step 8



Step 9



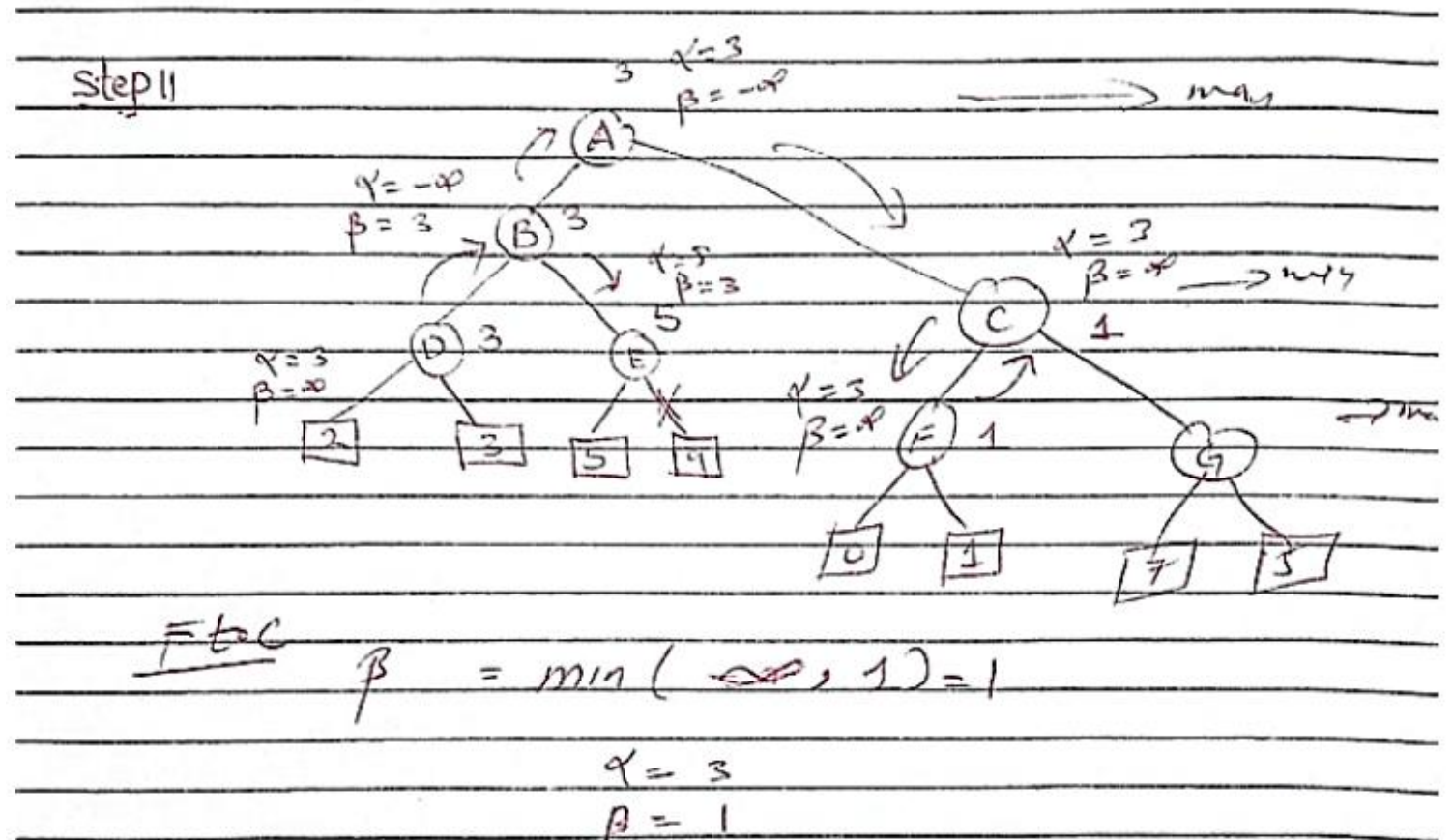
Step 9



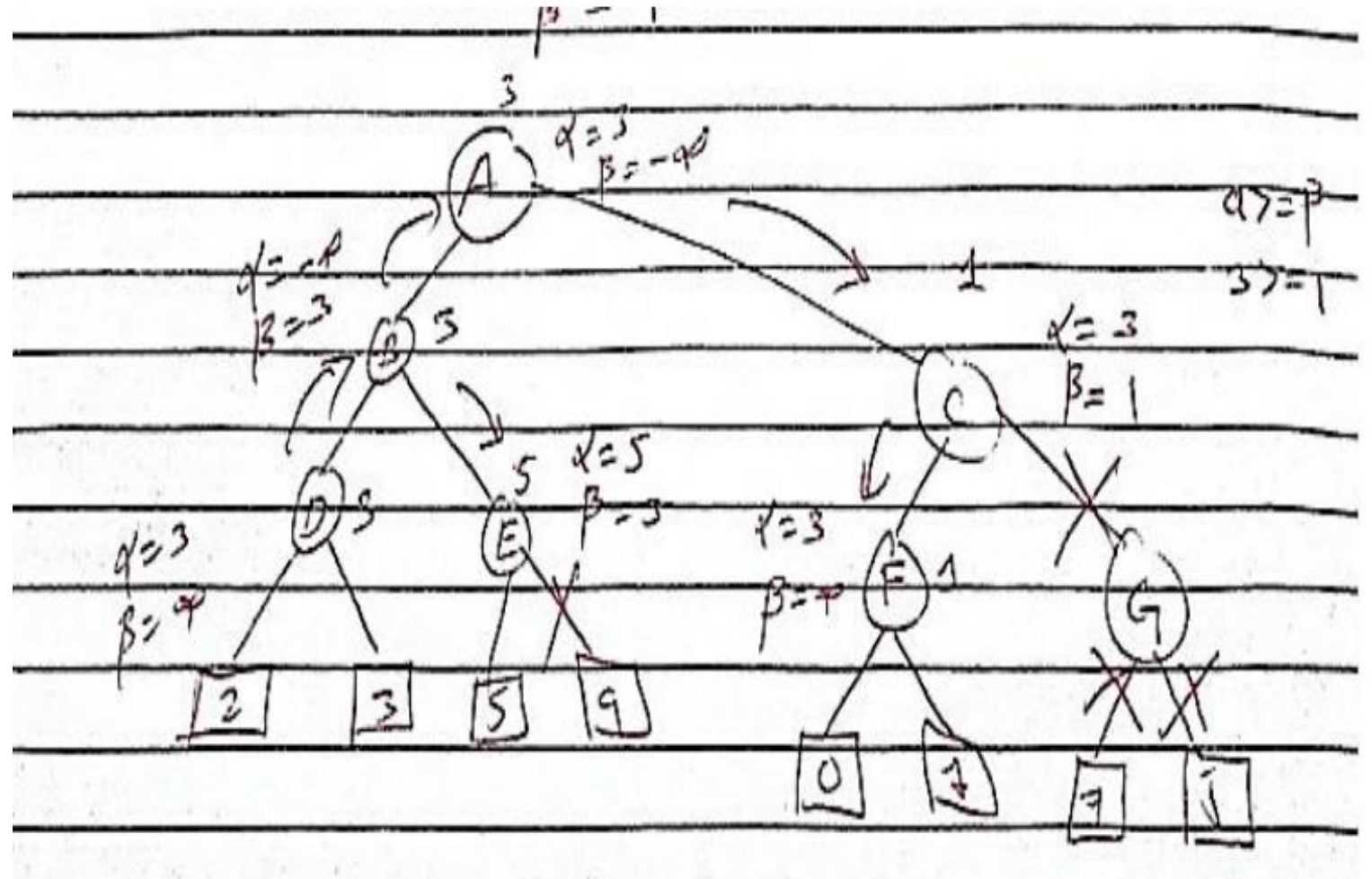
- Step 10

C to F
 $\max(0, 3) = 3$
 $\max(1, 3) = 3$

Step 11



Step 12



Move Ordering in AlphaBeta pruning:

- The effectiveness of alpha-beta pruning is highly dependent on the order in which each node is examined. Move order is an important aspect of alpha-beta pruning. It can be of two types:
- Worst ordering
- Ideal ordering

Worst ordering:

- In some cases, alpha-beta pruning algorithm does not prune any of the leaves of the tree, and works exactly as minimax algorithm. In this case, it also consumes more time because of alpha-beta factors, such a move of pruning is called worst ordering. In this case, the best move occurs on the right side of the tree.

Ideal ordering:

- The ideal ordering for alpha-beta pruning occurs when lots of pruning happens in the tree, and best moves occur at the left side of the tree.

Pseudocode for Mini-Max

```
Function Mini-Max(node, depth, Maximizing-Player) is
    if depth==0 or node is a terminal node then
        return the heuristic value of node
    if maximizing-Player then
        value =-infinity
        for each child of node do
            value := max(value, mini-max(child, depth-1, node))
        return value
    else
        value = +infinity
        for each child of node do
            value= min(value, mini-max( child, depth-1, True))
        return value
```


Pseudocode for Mini-Max with Alpha- Beta Pruning

```
function Alpha_beta(node, depth, alpha, beta, maximizingPlayer) is
    if depth == 0 or node is a terminal node then
        return the heuristic of node
    if MaximizingPlayer then
        max_value = -infinity
        for each child of node do
            value = max(child, depth-1, alpha, beta, False)
            max_value = max(max_value, value)
            alpha = max(alpha, value)
            if beta <= alpha
                break
        return max_value
    else
        min_value = +infinity
        for each child of node do
            value = mini-max(child, depth-1, alpha, beta, true)
            min_value = min(min_value, value)
            beta = min(beta, value)
            if beta <= alpha
                break
        return min_value
```